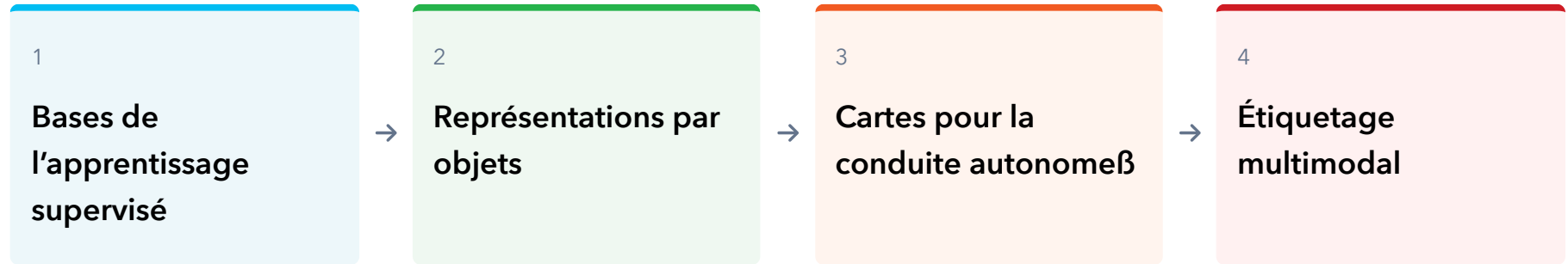


 Loading slide...

Learning a representation useful to the robot



Understand the data, losses and computations that link an observation to an exploitable representation.

Course objective

Be able to explain how a model learns, interpret its errors, and integrate its outputs into a geometric perception system useful for robotic planning.

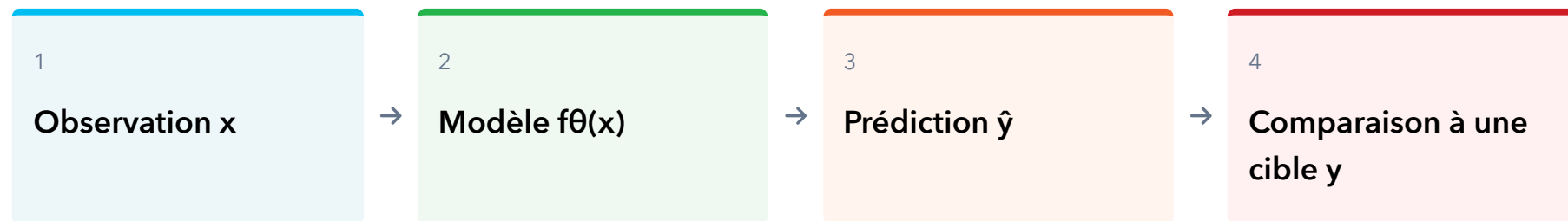
Plan

- Foundations of Supervised Learning
- Object-Based Representations
- Maps for Autonomous Driving
- Multimodal Supervision
- Synthesis

Foundations of Supervised Learning

How do we turn annotated examples into a model that generalizes?

One observation, one target, one prediction



A robot receives an image. The target can be a **distance**, a **class**, a **box**, or a **pose**.

The parameters θ are adjusted on labeled examples.

Example

For an actual distance of 8 m, the model predicts 6 m. The loss function quantifies this error. The error then guides the correction of the model's parameters.

Learning by minimizing a cost function

For an annotated dataset $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^N$:

$$\theta^* = \arg \min_{\theta} L(\theta), \quad L(\theta) = \frac{1}{N} \sum_{i=1}^N \ell(f_{\theta}(x_i), y_i).$$



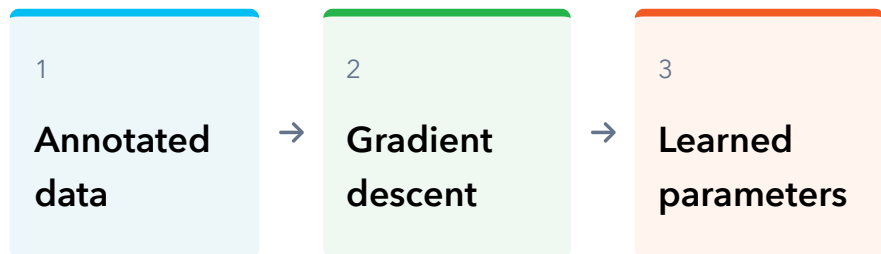
Key takeaway

Minimizing the loss function $L(\theta)$ on the training dataset allows learning the parameters θ of the prediction function f :

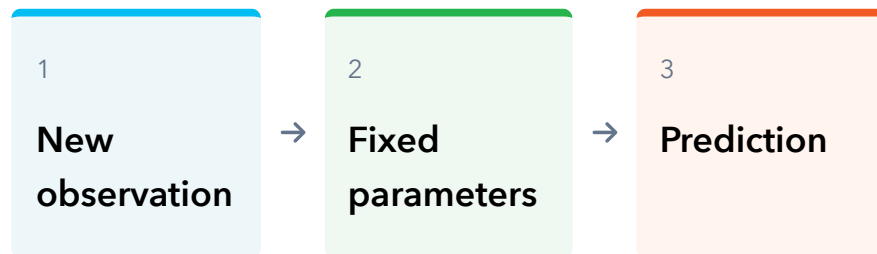
$$f_{\theta}(x_i) = \hat{y}_i$$

Train then use the model

Learn a model:

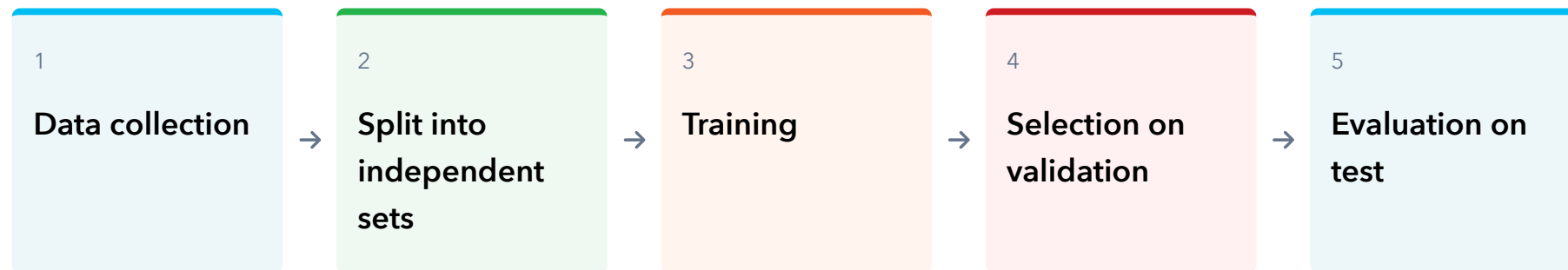


Use a model:



Split training, validation, and test sets

Set	Role	What can depend on it
Training	Compute the loss and gradients	Network parameters
Validation	Compare model choices	Hyperparameters, stopping, selection
Test	Evaluate the chosen model	Final performance report



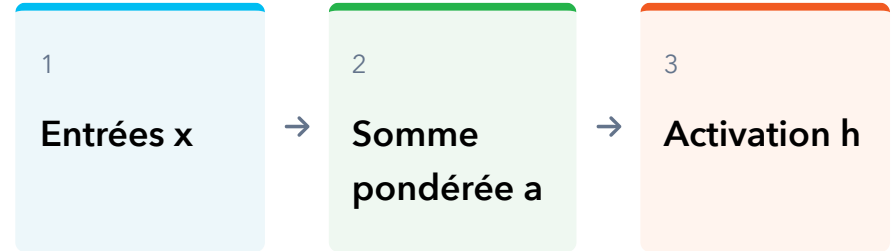
Avoiding contamination

In robotics, split by trajectory, location, or session. Two consecutive frames from a video are strongly correlated.

The neuron: a transformation followed by an activation

$$a = w^\top x + b, \quad h = \tanh(a).$$

- w weighs the inputs; b is a bias.
- a is a preactivation.
- Non-linear activations make it possible to learn more complex functions.



A network composes several transformations

$$h^{(1)} = \phi(W_1x + b_1), \quad h^{(2)} = \phi(W_2h^{(1)} + b_2), \quad \hat{y} = g(W_3h^{(2)} + b_3).$$



The **weights** and **biases** constitute the parameters. An intermediate feature can describe a texture, an edge, or a structure.

An encoder transforms an image or a point cloud into features. The last layer of a network (head) estimates a class, a box, a descriptor, etc from the features.

Regression: predicting a continuous quantity

The target is numeric: depth, position, dimensions, or velocity.

$$e = \hat{y} - y, \quad \ell_{\text{quad}} = \frac{1}{2}e^2, \quad \frac{\partial \ell}{\partial \hat{y}} = e.$$

The gradient indicates which direction to correct the prediction.

Worked example

True distance: 8 m. Prediction: 6 m.

$$e = -2 \text{ m}, \ell = \frac{1}{2} \cdot (-2)^2 \text{ m}^2 \text{ and } \partial \ell / \partial \hat{y} = -2 \text{ m}.$$

Descending the gradient **increases** the prediction $\hat{y}$ to reduce the error.

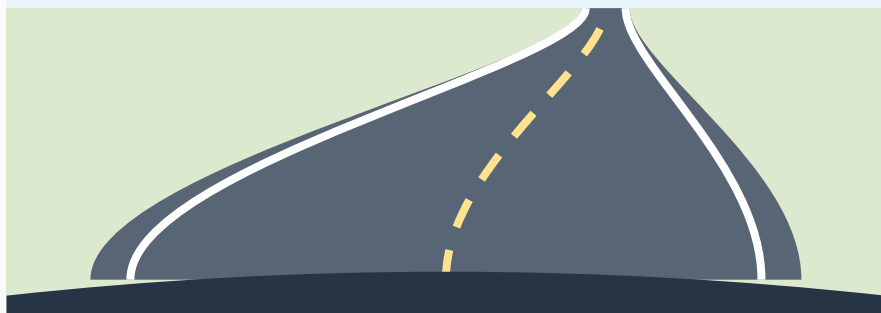
Regression example: from an image to a steering angle

Imitation driving example · scenes and illustrative values

Réinitialiser la démonstration

Entrée x · image de la caméra avant

Route, marquages et courbe à venir



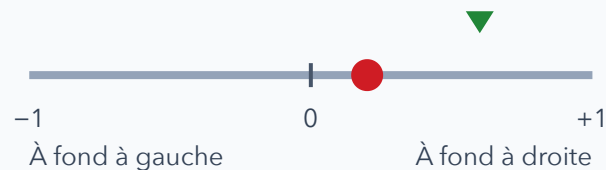
Virage à gauche

Route droite

Virage à droite

Sortie $\hat{y}$ · un seul nombre continu

$\hat{y} = 0,20$



Prédiction $\hat{y}$

▲ Cible enregistrée avec l'image : $y = 0,60$

Erreur $\hat{y} - y = -0,40$

Perte $\frac{1}{2}(\hat{y} - y)^2 = 0,080$

A 2D image $\rightarrow$ a scalar. Here, the angle is normalized between -1 and $+1$; it could also be expressed in radians.

Classification: from scores to probabilities

For C classes, the network predicts scores $s_1, \dots, s_C$ called logits:

$$p_c = \frac{\exp(s_c)}{\sum_{k=1}^C \exp(s_k)}, \quad \sum_c p_c = 1.$$



Classifying a region observed by the robot

In the order **road, pedestrian, vehicle**, these scores give 66.5%, 24.5% and 9.0%. The model chooses "road". If the annotation is "pedestrian", this prediction is incorrect.

Key takeaway

Scores can be negative. Probabilities are positive and normalized. The largest score remains the most probable class.

Cross-entropy: the probability of the correct class

A region contains a **pedestrian**. What probability does the model assign to this annotation?

Class c	Target y_c	Prediction p_c
Road	0	0.6652
Pedestrian	1	0.2447
Vehicle	0	0.0900

The **one-hot** label is 1 for the correct class, 0 for the others. The probabilities p_c come from the softmax.

Cross-entropy compares the target y to the predicted distribution p :

$$\ell_{\text{CE}} = - \sum_{c=1}^C y_c \ln p_c.$$

Only the term for the annotated class c^* remains:

$$\ell_{\text{CE}} = - \ln p_{c^*}.$$

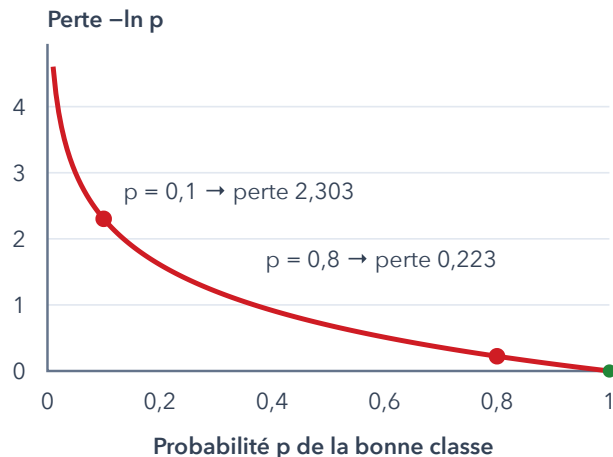
The target selects the "pedestrian" term

$$\ell = -[0 \ln p_{\text{road}} + 1 \ln p_{\text{pedestrian}} + 0 \ln p_{\text{vehicle}}] = -\ln(0.2447) \approx 1.408.$$

Minimizing this loss encourages the model to assign **more probability to the pedestrian**.

Why a logarithmic loss?

With $p = p_{c^*}$, the loss $-\ln p$ decreases as the probability of the correct class increases.



Two correct predictions, two losses

The target is "pedestrian"; order: road, pedestrian, vehicle.

Predicted probabilities	Loss
(0.4; 0.5; 0.1)	0.693
(0.05; 0.9; 0.05)	0.105

The pedestrian comes first in both cases, but the second prediction gives it more probability.

At $p = 1$: zero loss. If $p \rightarrow 0$: loss $\rightarrow +\infty$.

What we are training

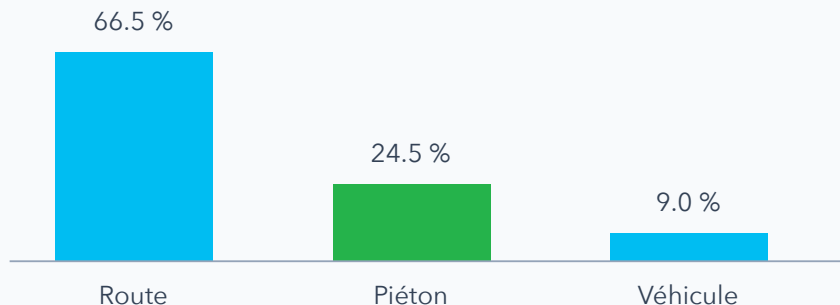
The loss evaluates the **probability of the annotation**, beyond a simple correct/incorrect verdict. On a mini-batch, we take the average of the losses of the examples.

Manipulating scores and cross-entropy

Expérience pédagogique · données synthétiques

Réinitialiser la démonstration

Probabilités : somme = 1



Score Route : 2.0



Score Piéton : 1.0



Score Véhicule : 0.0



Piéton ▼

Cible : **Piéton**

Étiquette $y = [0 ; 1 ; 0]$

$p(\text{cible}) = \mathbf{0.2447}$

$\ell = -\ln p(\text{cible}) = \mathbf{1.408}$

Gradients des scores : $p - y$

0.665 ; -0.755 ; 0.090

Target "pedestrian": increasing its score decreases the loss. Then change the target to compare.

Chain rule connects the loss to the parameters

Changing a weight changes an activation, then the prediction, then the loss. The chain rule measures **how this effect propagates**.

$$a = w_1x + b_1, \quad h = \tanh(a), \quad \hat{y} = w_2h + b_2, \quad \ell = \frac{1}{2}(\hat{y} - y)^2.$$

$$\frac{\partial \ell}{\partial w_1} = (\hat{y} - y) \times w_2 \times (1 - h^2) \times x.$$

$\hat{y} - y$	w_2	$1 - h^2$	x
Slope of the loss	Effect of h on $\hat{y}$	Slope of the activation	Effect of w_1 on a

Backpropagating

The forward pass provides $a, h, \hat{y}$. The backward pass multiplies the local derivatives to obtain the sensitivity of the loss to each parameter.

Backpropagation: one weight, one computation, one effect

Steering angle example: $x = 1$, $y = 0.3$, with $w_1 = b_1 = 0$, $w_2 = 0.5$ and $b_2 = 0.1$.

Forward computation	Value	Local derivative for the backward pass
$a = w_1x + b_1$	0	$\partial a / \partial w_1 = x = 1$
$h = \tanh(a)$	0	$\partial h / \partial a = 1 - h^2 = 1$
$\hat{y} = w_2h + b_2$	0.1	$\partial \hat{y} / \partial h = w_2 = 0.5$
$\ell = \frac{1}{2}(\hat{y} - y)^2$	0.02	$\partial \ell / \partial \hat{y} = -0.2$

Which weight should be increased?

$$\partial \ell / \partial w_1 = (-0.2) \times 0.5 \times 1 \times 1 = -0.1.$$

With $\eta = 0.2$, $w_1' = 0 - 0.2(-0.1) = 0.02$. Changing only this weight, $\hat{y}' \approx 0.110$: the angle gets closer to the target 0.3

.

Gradient descent: choosing a direction and a step size

$$\theta_{k+1} = \theta_k - \eta \nabla_{\theta} L(\theta_k).$$

- The gradient points toward the steepest local increase.
- The minus sign gives a descent direction.
- $\eta > 0$ controls the step length.

A numerical step

$$L(w) = \frac{1}{2}(w - 3)^2.$$

$$\text{At } w = 1, \nabla L = -2.$$

$$\text{With } \eta = 0.2: w' = 1 - 0.2(-2) = 1.4.$$

The loss goes from 2 to 1.28.

Full-batch, SGD and mini-batches

A mini-batch B contains $|B|$ examples. We **average their gradients**, then perform an update:

$$g_B = \frac{1}{|B|} \sum_{i \in B} \nabla_{\theta} \ell_i, \quad \theta \leftarrow \theta - \eta g_B.$$

Method	Examples per update	For 3,200 images, in one epoch
Full batch	All examples	1 update
SGD, one example at a time	1	3,200 updates
Mini-batch	32 in this example	100 updates

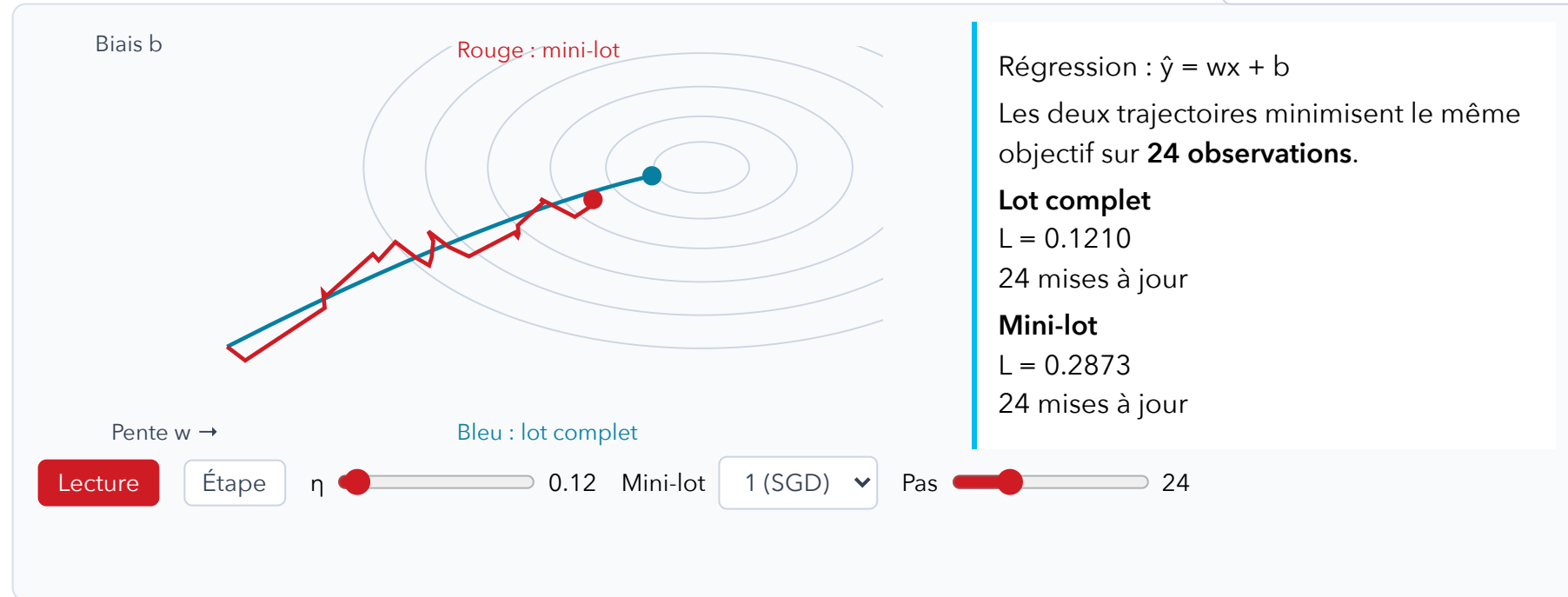
Two examples don't always push in the same direction

For the same weight, two images give gradients -2 and $+1$. The mini-batch gives $g_B = (-2 + 1)/2 = -0.5$: with $\eta = 0.2$, the weight increases by 0.1 .

Comparing optimization trajectories

Expérience pédagogique · données synthétiques

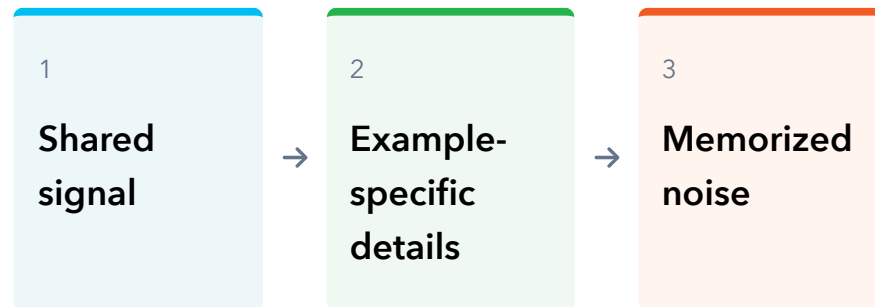
Réinitialiser la démonstration



Same objective and same starting point. Change η then the mini-batch size; observe stability, noise and speed.

When learning more degrades generalization

A very flexible model can learn details specific to the examples and to annotation noise. Its training loss then keeps decreasing, while its validation loss increases.



Overfitting

Overfitting is diagnosed by the generalization gap, not solely by a low training loss.

Overfitting is like over-memorizing

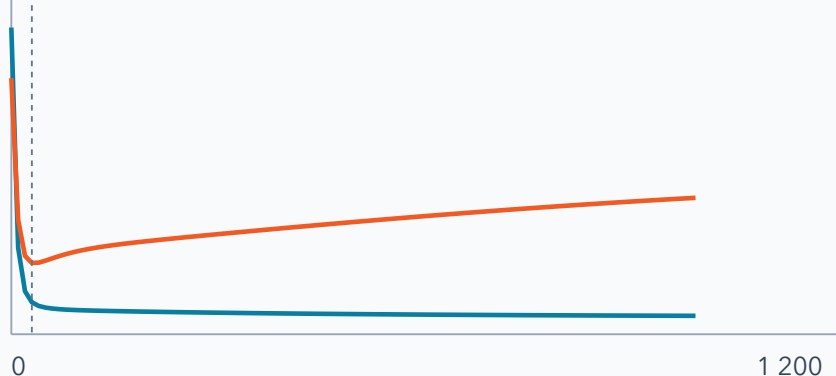
The model associates a red building with a right turn. It predicts angles well on this route, but gets it wrong on an unfamiliar road where a red building precedes a left turn.

Choosing the model with validation

Expérience pédagogique · données synthétiques

Réinitialiser la démonstration

Erreur quadratique moyenne
Meilleure validation



Entraînement : 0.034

Validation : 0.249

Test masqué pendant le choix du modèle.
Arrêt conseillé : époque 30.

Capacité 16 Époques 1000
Bruit 0.55 λ 0.000

Choisir la meilleure validation

Figier puis révéler le test

Explore capacity, noise and regularization. Choose an epoch, then freeze the model before revealing the test.

Regularize: limit solutions that are too fragile

The goal of regularization is to prevent overfitting and ensure generalization.

Example of regularization: L2 penalty on the network weights:

$$L_{\text{reg}}(\theta) = \underbrace{\frac{1}{N} \sum_i \ell_i}_{\text{data fit}} + \underbrace{\lambda \sum_j \theta_j^2}_{\text{penalty on weights}} .$$

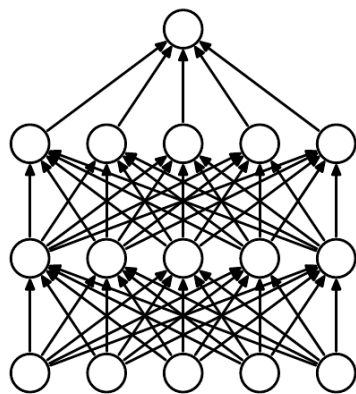
Two solutions give the **same data loss** 0.10. With $\lambda = 0.01$:

Weights	$\ \theta\ ^2$	Penalty	Total cost
$\theta_A = (3, 4)$	25	0.25	0.35
$\theta_B = (1, 2)$	5	0.05	0.15

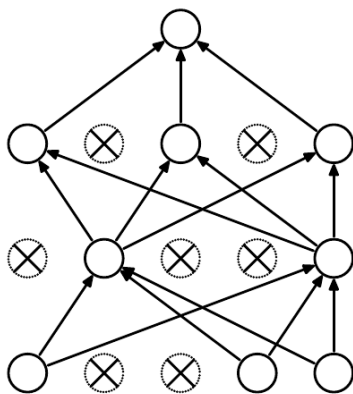
A preference, to be tuned on validation

Here the objective prefers B. However, a λ that is too large can prevent learning the signal: a low norm does not guarantee good generalization.

Dropout Regularization: Learning with Sub-networks



(a) Standard Neural Net



(b) After applying dropout.

During training, each activation is kept with probability $1 - p$.

$$\tilde{h} = \frac{mh}{1-p}, \quad m \sim \text{Bernoulli}(1-p).$$

At inference, generally, all activations are used: $\tilde{h} = h$.

Dropout reduces the number of activations and parameters affected during training. General rule: fewer parameters for the same amount of data reduces the risk of overfitting.

Srivastava et al., JMLR 2014, Figure 1. [Paper and source of the figure.](#)

Augmenting Robotics Data

Observation	Useful transformations	Consistency to preserve
Color image	Lighting, contrast, blur, occlusion	Class identity and physical boundaries
Image + boxes	Cropping, translation, resizing	Boxes and masks transformed together
RGB-D / multiview	Transformations consistent across sensors	Depth, intrinsics, correspondences
LiDAR point cloud	Rotation about the vertical axis, noise, point removal	Boxes and poses in the same frame

Validity of examples

An augmentation must represent a plausible variation of the task. An arbitrary rotation of gravity can produce an impossible scene and hurt training.

A complete learning loop



```
for x, y in training_loader:
    x, y = augment_together(x, y)
    optimizer.zero_grad()
    prediction = model(x)
    loss = criterion(prediction, y)
    loss.backward()
    optimizer.step()
```

Image-angle example: a mini-batch contains 32 RGB images (x : $32 \times 3 \times H \times W$) and 32 angles (y : 32×1). The model predicts one angle per image.

Validation in evaluation mode, without gradients; save the best model according to the chosen criterion.

Object-Based Representations

Detect, locate, orient, and organize the objects in a scene.

What should an object representation contain?

Output	Information	Robotic use
Class + 2D box	Category and region in the image	Region selection, visual tracking
3D box	Center, dimensions, orientation	Distance, footprint, collision
Pose of a known object	Transformation from model to sensor	Grasping, assembly, inspection
Node in a map	Persistent identity and relationships	Object search, semantic navigation



2D Detection: Training Data

An image can contain a variable number of objects:

$$Y = \{(c_j, b_j)\}_{j=1}^M, \quad b_j = (u_j, v_j, w_j, h_j).$$

For each object, c_j is the class; (u_j, v_j) is the **top-left corner** and (w_j, h_j) the width and height, in pixels.

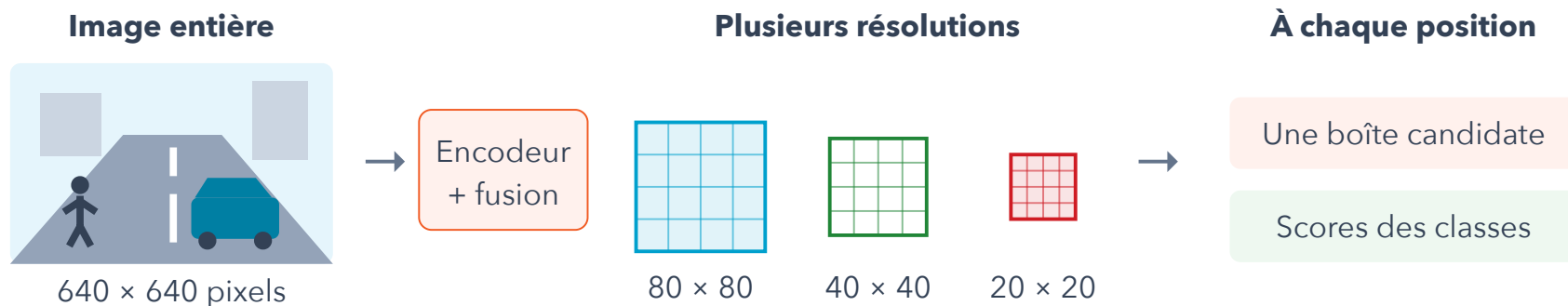
Example

Example: an image annotated with two cars and a pedestrian. The detector must predict their classes, positions, and dimensions in the image.



YOLO: detect in a single pass

YOLO – You Only Look Once predicts classes and boxes from the entire image, in a single pass of the network.
Example presented: **YOLOv8**.

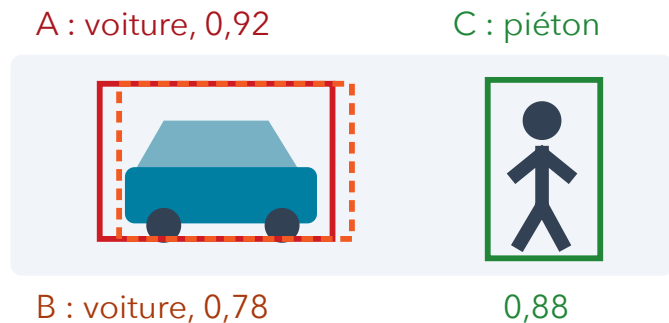


At 640 × 640 pixels, the three maps give $80^2 + 40^2 + 20^2 = 8\,400$ prediction positions. The different resolutions help detect objects of varying sizes.

Redmon et al., YOLO, CVPR 2016; modern example: Ultralytics YOLOv8. Pedagogical diagram.

YOLO: keeping the useful detections

At inference, YOLOv8 can propose several boxes for the same object. Two steps reduce these candidates: **score threshold**, then **duplicate suppression (NMS)**.



Thresholds: score 0.50; IoU 0.50.

Candidate	Class	Score
A	Car	0.92
B	Car	0.78
C	Pedestrian	0.88
D, not drawn	Car	0.20

NMS keeps the box with the best score, then discards boxes of the same class that overlap it too much. It repeats this choice among the remaining candidates.

Which boxes to keep?

D is discarded by the score threshold. Since $\text{IoU}(A, B) \approx 0.85 > 0.50$, A is kept and B is removed. This leaves **A and C**.

NMS: non-maximum suppression; IoU: intersection over union. YOLOv8 post-processing. The thresholds are for teaching purposes.

One loss for the class, one loss for the box

Once each prediction is matched to its target, we correct **what the object is** and **where it is located**:

$$L_{\text{det}} = \lambda_{\text{cls}} L_{\text{cls}} + \lambda_{\text{box}} L_{\text{box}} + \lambda_{\text{IoU}} L_{\text{IoU}}.$$

Term	What it compares	Example error
Classification	Probabilities and target class	A car classified as a pedestrian
Box regression	Predicted and annotated coordinates	A box offset by 40 pixels
Overlap	Regions covered by the boxes	A box that is too large despite a good center

The λ weights balance the contributions

The terms have different scales. Their weighting and exact form depend on the detector. A perfect box can still have the wrong class.

IoU: how much surface do the boxes share?

Intersection over union compares the shared part to the whole region covered:

$$\text{IoU}(A, B) = \frac{|A \cap B|}{|A \cup B|} = \frac{|A \cap B|}{|A| + |B| - |A \cap B|}.$$

Two boxes of 100×100 pixels, at the same height, offset horizontally by 40 pixels:

Quantity	Calculation	Area in square pixels
Intersection	$(100 - 40) \times 100$	6 000
Union	$10\,000 + 10\,000 - 6\,000$	14 000

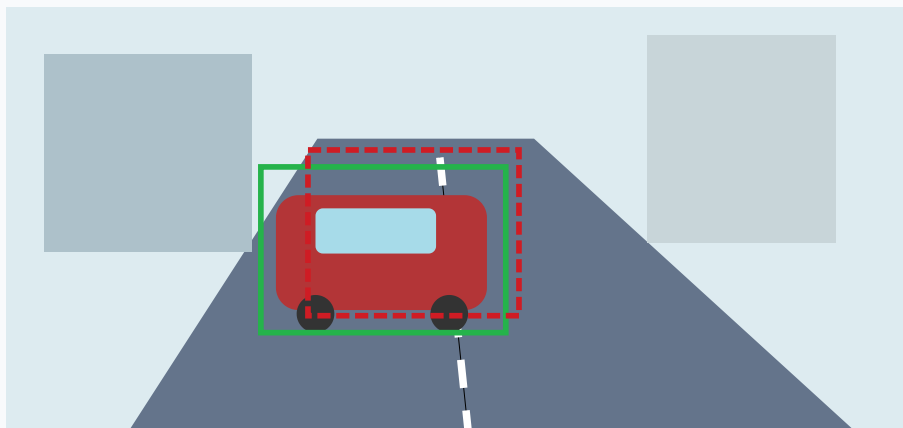
A visible overlap can still be insufficient

$\text{IoU} = 6\,000 / 14\,000 \approx 0.43$. At an evaluation threshold of 0.50, this localization is not enough, even if the class is correct.

IoU Visualization

Expérience pédagogique · données synthétiques

Réinitialiser la démonstration



Vert : annotation

Rouge : prédiction

x y Largeur Score

Classe cible : **voiture**

$p(\text{voiture}) = 0.818$

Intersection / union :

IoU = 0.638

$\ell_{\text{classe}} = 0.201$

$\ell_{\text{boîte}} = 0.051$

Somme illustrative : **0.253**

Make the boxes coincide, then make the class wrong: the two errors are controlled separately.

Evaluating detections at the right level

A correct detection combines a correct class with sufficient overlap with an annotation. A single annotation must not validate multiple predictions.

$$\text{precision} = \frac{TP}{TP + FP}, \quad \text{recall} = \frac{TP}{TP + FN}.$$

TP : true positives; FP : false positives; FN : false negatives.

From score to performance

Average precision, AP, summarizes the precision-recall curve. Always specify the classes and overlap thresholds of the protocol.

Three annotated pedestrians, four detections

Two pedestrians are correctly detected, one is missed; one box is a duplicate and another targets a pole. So $TP = 2$, $FP = 2$, $FN = 1$: **precision = 50%, recall $\approx$ 67%.**

From Confidence Threshold to the Precision-Recall Curve

For a fixed class and IoU threshold, sort detections by decreasing score. Example: **three annotated pedestrians**.

Score	Detection added	Cumulative precision	Cumulative recall
0.95	Pedestrian 1: TP	$1/1 = 100\%$	$1/3$
0.90	Pole: FP	$1/2 = 50\%$	$1/3$
0.80	Pedestrian 2: TP	$2/3 \approx 67\%$	$2/3$
0.60	Duplicate of pedestrian 1: FP	$2/4 = 50\%$	$2/3$
0.40	Pedestrian 3: TP	$3/5 = 60\%$	$3/3$

Lowering the threshold lets more detections in. Recall increases or stays constant; precision can go up or down.

AP summarizes the precision-recall curve following the interpolation protocol. It is not the average of the five precisions in the table.

Fictional example, annotations without "crowd" or ignored cases. COCO reference protocol.

Varying the detection threshold

Synthetic image · fixed scores · class: pedestrian · no NMS

Réinitialiser la démonstration

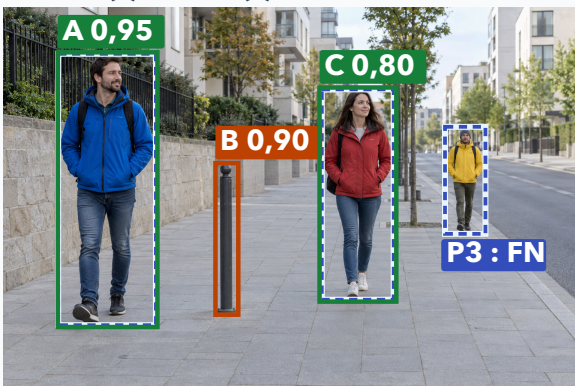
Seuil de score τ  0,80

Balayer le seuil

☒ Annotations

IoU d'évaluation $\geq 0,50$

Image : 3 détection(s) retenue(s)



 VP : piéton retrouvé  FP : erreur ou doublon  Pointillés bleus : annotations Courbe brute · lettres = dernières boîtes ajoutées

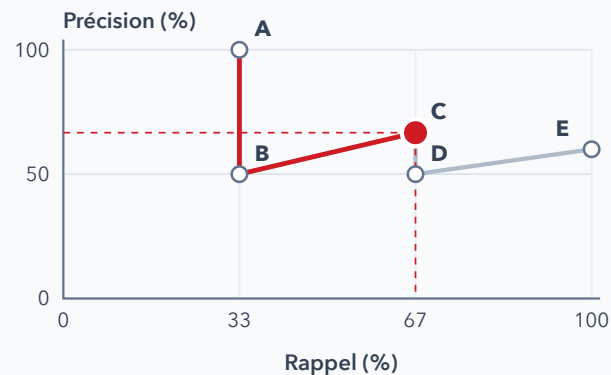
VP 2 · FP 1 · FN 1

Précision : $2/3 = 66,7 \%$

Rappel : $2/3 = 66,7 \%$

C retrouve un autre piéton : le rappel et la précision augmentent.

Courbe précision-rappel : point au seuil courant



Lower the threshold or click on the curve. Between two scores, the detections and the point remain unchanged.

RGB-D: from image to metric points

The pixel (u, v) and its axial depth $d = D(u, v)$ give a point in the camera frame:

$$X_C = \begin{bmatrix} (u - c_x)d/f_x \\ (v - c_y)d/f_y \\ d \end{bmatrix} = dK^{-1} \begin{bmatrix} u \\ v \\ 1 \end{bmatrix}.$$

Subtract the principal point (c_x, c_y) , divide by the focal lengths (f_x, f_y) , then multiply by the depth: pixels become meters.

A point on a box to grasp

$f_x = f_y = 500$ px, $(c_x, c_y) = (320, 240)$, $(u, v) = (370, 240)$ and $d = 2$ m give $X_C = (0.20; 0; 2)$ m: **20 cm to the right** of the optical axis.

For the vehicle: $X_V = R_{V \leftarrow C} X_C + t_{V \leftarrow C}$. The extrinsic calibration fixes this frame change.

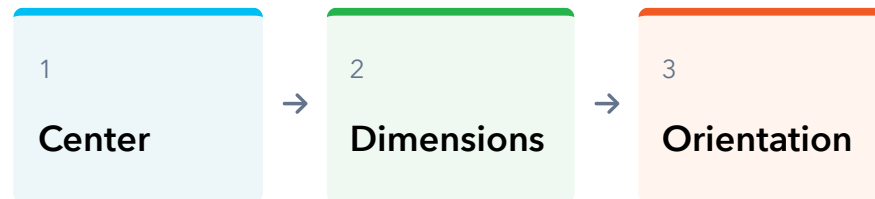
Parameterizing a 3D box

For objects assumed to be vertical:

$$b = (c_x, c_y, c_z, l, w, h, \psi).$$

- c : center, in meters.
- l, w, h : dimensions.
- ψ : orientation about the vertical axis.

A general orientation requires a 3D rotation.



Physical assumption

A vehicle box only needs a single yaw angle, since the car is assumed to have all four wheels on the ground. On the other hand, a tool that can be held in any orientation requires three rotational degrees of freedom.

A LiDAR versus camera detection



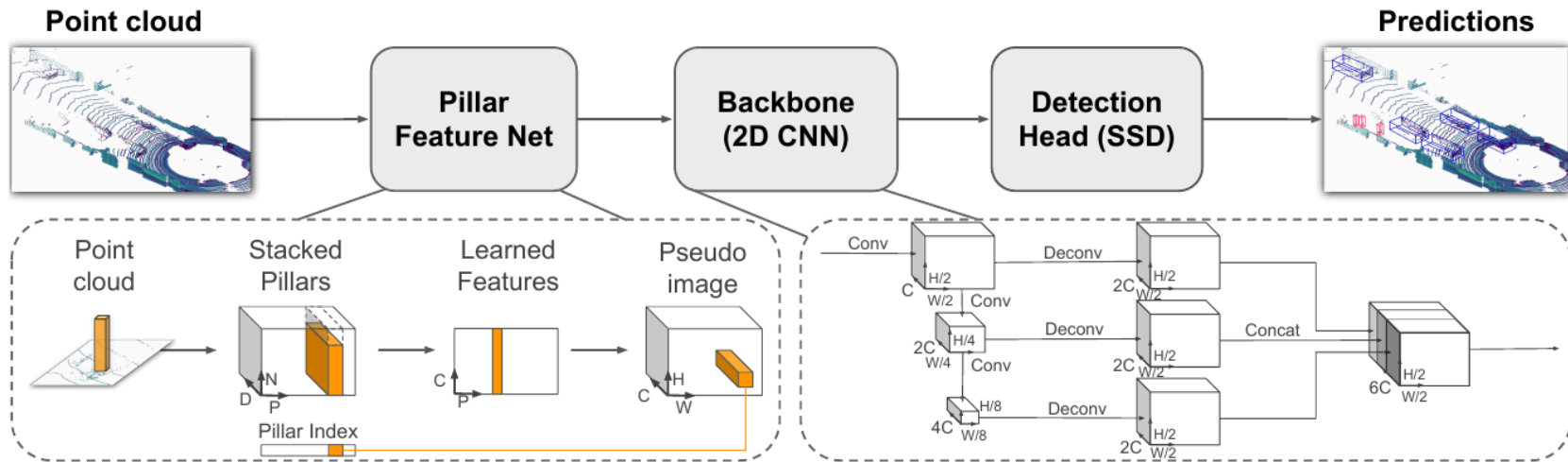
Top: point clouds and boxes in bird's-eye view.

Bottom: reprojection of the boxes into the images.

The images help here to **visualize** the result; PointPillars uses the LiDAR for this detection.

Lang et al., PointPillars, CVPR 2019, Figure 3. [Paper and figure source.](#)

PointPillars: from point cloud to pseudo-image



1. Group the points into vertical pillars.
2. Encode the points in each pillar.
3. Arrange the vectors in a grid.
4. Predict classes and 3D boxes.

The grid makes processing with 2D convolutions easier.

Lang et al., PointPillars, CVPR 2019, Figure 2. [Paper and figure source.](#)

Images and LiDAR: complementary information

Criterion	Images	LiDAR
Appearance and categories	Rich texture and color	Geometry, possibly intensity
Metric scale	Ambiguous in monocular	Direct distance measurement
Density	Dense in pixels	Sparse, especially at long range
Difficult conditions	Sensitive to lighting	Also sensitive to rain, fog, and occlusions
Fusion	Calibration and synchronization needed	Same requirement for coherent fusion

Design decision

Choose based on the sensor, the task, and the deployment conditions. Poorly calibrated fusion can degrade an estimate.

A pedestrian is waiting at the roadside

The image helps recognize the pedestrian; the LiDAR returns on their body provide their distance. The fusion must associate the pixels and points of the same pedestrian, at the same instant.

Estimating the pose of a known object

For several applications, it is necessary to estimate the pose of an object.

The simplest case is when the geometry of the object is known a priori. For example, we can have a CAD model of the object.

The CAD model defines the points on the object in the object frame O . The pose expresses them in the camera frame C :

$$X_C = \underbrace{R_{C \leftarrow O} X_O}_{\text{rotate the object's axes}} + \underbrace{t_{C \leftarrow O}}_{\text{place its origin}} .$$

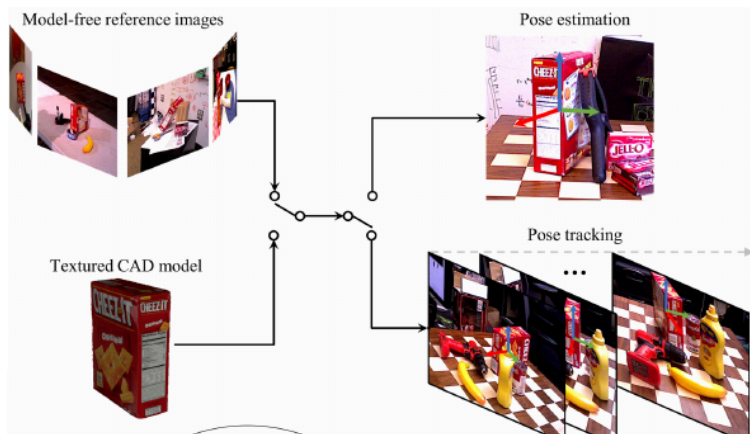
Six degrees of freedom: three for translation and three for rotation. The dimensions of the object are provided by its model.

Where is a point on the handle?

A point on the drill is $X_O = (0.1; 0; 0) \text{ m}$. A 90° rotation around z gives it as $(0; 0.1; 0)$.

With $t = (0.5; 0; 2) \text{ m}$, it becomes $X_C = (0.5; 0.1; 2) \text{ m}$. This position allows the robot to prepare its grasp.

A CAD model provided at inference time



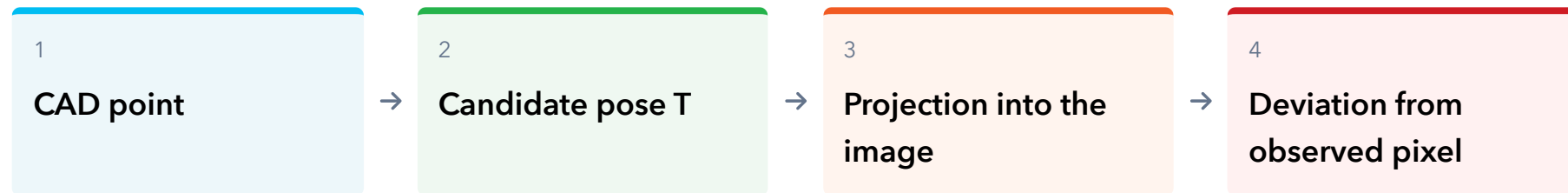
1. We look for the object in the image and determine its position and bounding box.
2. To obtain the orientation of the object, we match the points of the 3D (CAD) model with the points in the bounding box.
3. Once the points are matched, we can compute the orientation (e.g., in the manner of ICP and PnP).

Wen et al., FoundationPose, CVPR 2024, Figure 1, partie supérieure. [Article et source de la figure.](#)

Correspondences, then geometric estimation

For each point, we know its **3D position in the model** X_k^O and its **observed pixel** u_k . The pose T remains to be estimated:

$$\hat{T} = \arg \min_T \sum_k \left\| u_k - \underbrace{\pi(KTX_k^O)}_{\text{pixel predicted for this pose}} \right\|^2.$$



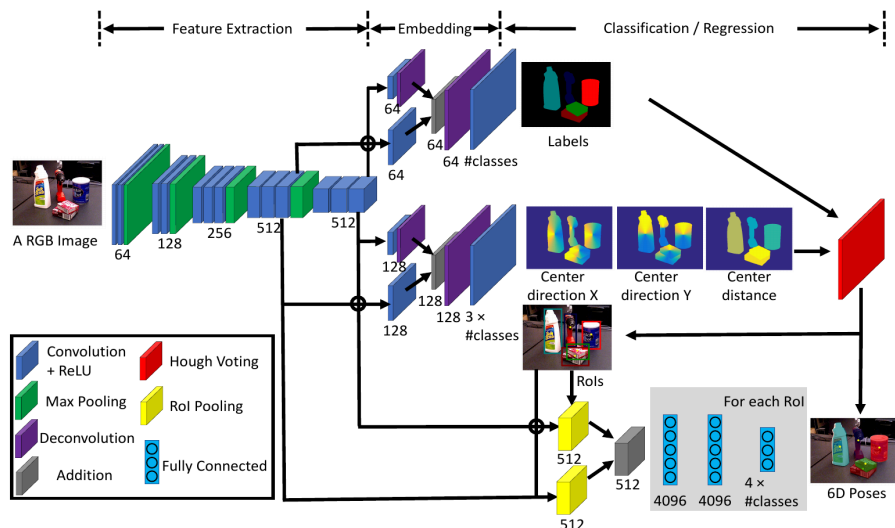
The residual is measured in the image

A corner observed at (320, 240) is projected at (324, 237): the residual is $(-4, 3)$ pixels.

Its contribution to the sum is $(-4)^2 + 3^2 = 25 \text{ px}^2$.

PnP estimates the pose from these correspondences; a refinement step reduces the reprojection error.

PoseCNN: predicting pose with annotated examples



Shared features feed three outputs: segmentation, translation, and rotation.

The training targets contain known poses. Rotation is regressed as a quaternion.

The model is specialized for the objects it was trained on.

Xiang et al., PoseCNN, RSS 2018, Figure 2. [Paper and figure source.](#)

Quaternions: normalization and double representation

A unit quaternion $q = (q_w, q_x, q_y, q_z)$ represents a rotation:

$$\|q\|_2 = 1, \quad R(q) = R(-q).$$

Normalize the network's output before forming the rotation matrix.

Sign ambiguity

A naive loss $\|\hat{q} - q\|^2$ can penalize two identical rotations.

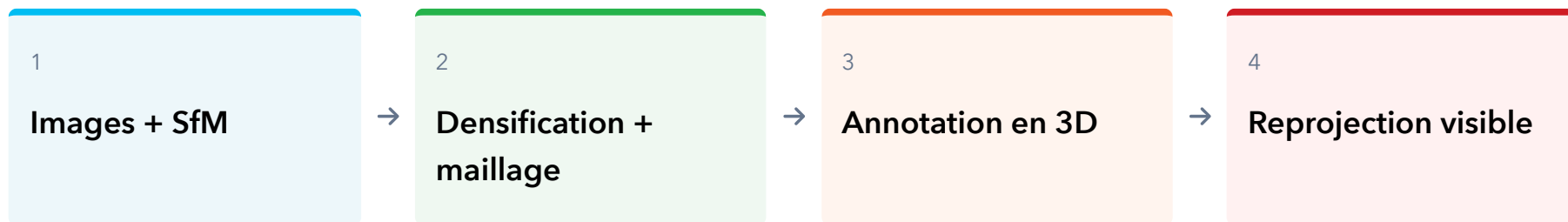
One option:

$$\min (\|\hat{q} - q\|^2, \|\hat{q} + q\|^2) .$$

Same orientation, artificial error

$q = (1, 0, 0, 0)$ and $\hat{q} = (-1, 0, 0, 0)$ both leave the object unrotated. Yet $\|\hat{q} - q\|^2 = 4$; the loss that accounts for sign equals 0.

Annotating a Reconstruction to Supervise Multiple Views

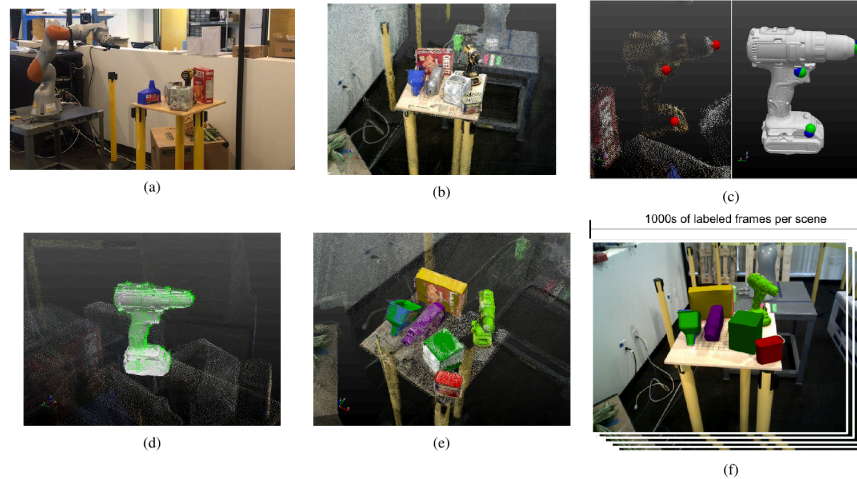


Structure from motion, **SfM**, estimates cameras and a geometry from images. **SfM** is the offline equivalent of SLAM, so it is not bound by real-time execution constraints. A dense reconstruction and a mesh then enable spatial annotation.

Saving on annotation

A single 3D annotation can serve many images. It requires consistent poses, a sufficiently static scene, and handling of occlusions.

Annotating a Reconstructed Scene



The pipeline collects an RGB-D sequence, reconstructs the scene, aligns object models, then reprojects their labels.

Marion et al., LabelFusion, ICRA 2018, Figure 1. [Paper and figure source.](#)

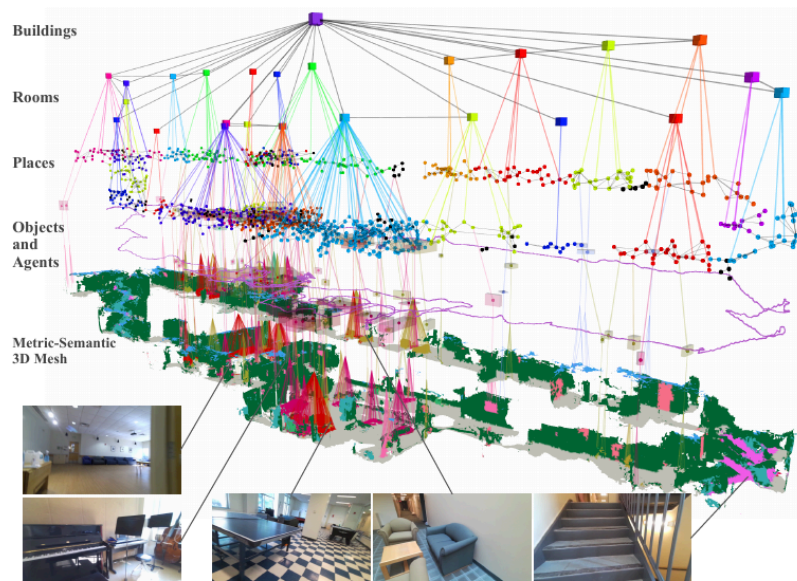
From objects to a hierarchical map

Scene graph 3: organizes geometry, objects, places and building in a single hierarchy.

The nodes (objects, places, rooms) have class labels and poses.

The relations represent membership, proximity and connectivity.

Example: "Find a chair in the meeting room." The graph links the chair to the room, then the room to the traversable places to reach it.

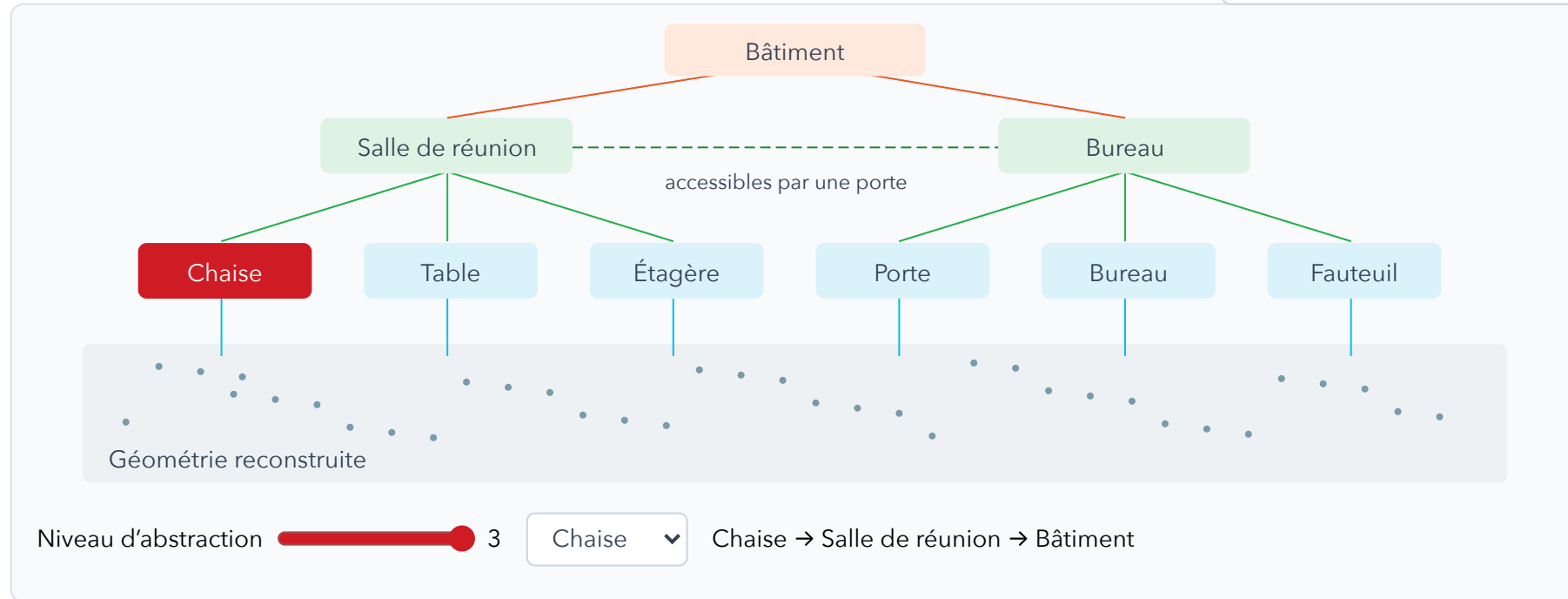


Hughes et al., Hydra, RSS 2022, Figure 1. [Paper and figure source.](#)

Explore the levels of a scene graph

Expérience pédagogique · données synthétiques

Réinitialiser la démonstration

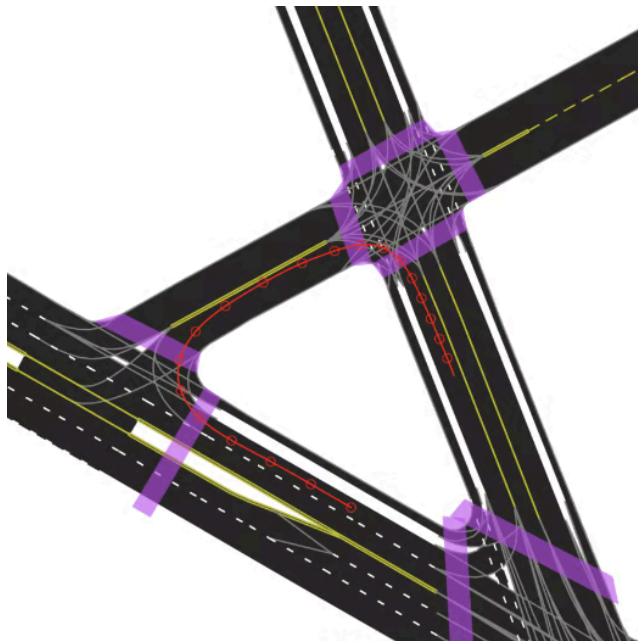


Change the level of abstraction and select an object to track its spatial membership.

Maps for Autonomous Driving

Build a geometric and topological representation the vehicle can use.

Maps for on-road planning



An excerpt of an HD map from Argoverse 2.

Wilson et al., Argoverse 2, NeurIPS Datasets and Benchmarks 2021. Illustration from the [official guide: Vector Map, Lane-Level Geometry](#).

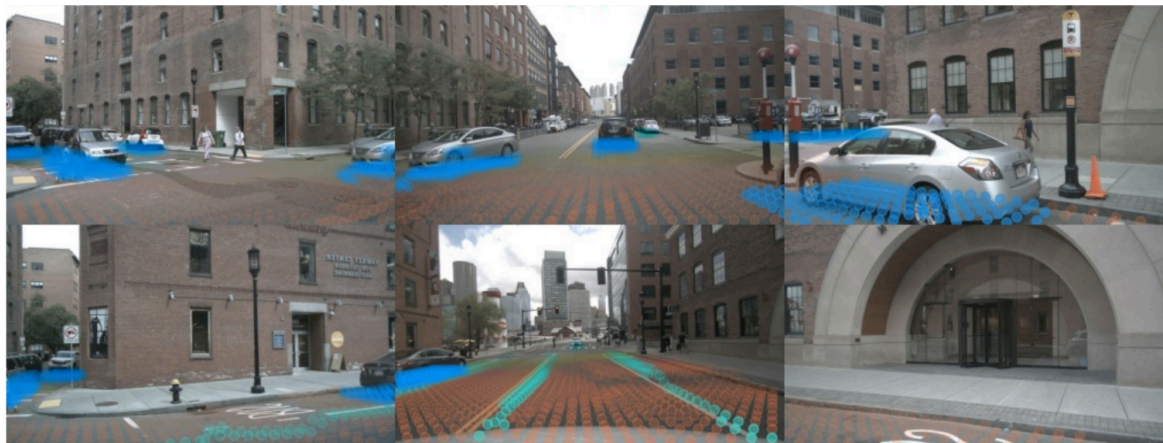
A high-definition map, **HD**, describes the lanes, their boundaries, crossings, and connections relevant to driving.

- **White / yellow:** lane markings and boundaries.
- **Purple:** pedestrian crossings.
- **Gray, at intersections:** implicit corridors between lanes.
- **Red:** vehicle trajectory, shown as a reference.

BEV: Bird's-Eye View

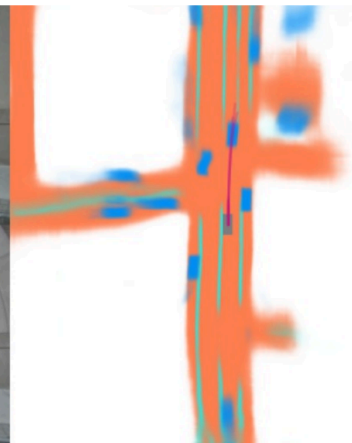
The bird's-eye view, **BEV** (*bird's-eye view*), brings observations together in a single frame on the ground, around the vehicle.

Six cameras · predictions reprojected as points



■ Vehicles ■ Drivable surface ■ Lanes

BEV segmentation



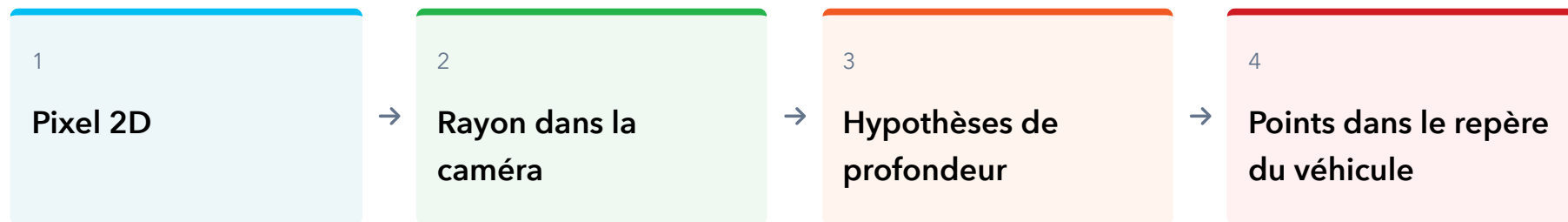
On the right, a **semantic prediction in the ground plane**: positions and distances around the vehicle can be compared.

Phillon and Fidler, Lift-Splat-Shoot, ECCV 2020, figure 1. [Paper and figure.](#)

BEV from image

A pixel $\tilde{u} = (u, v, 1)^\top$ defines a ray. For an axial depth d :

$$X_C(d) = dK^{-1}\tilde{u}, \quad X_V(d) = R_{V \leftarrow C}X_C(d) + t_{V \leftarrow C}.$$



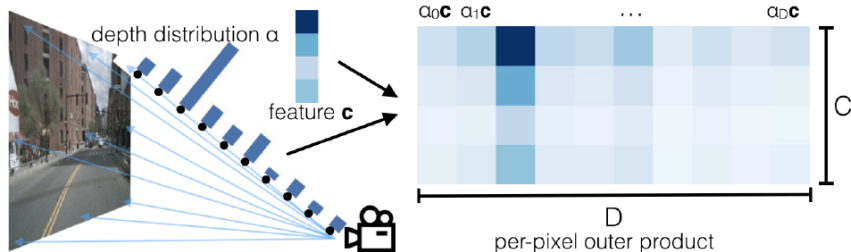
Ambiguity

Without depth or a surface hypothesis, a pixel does not determine a unique 3D point.

If the car is equipped with a LiDAR

We can estimate the depth of pixels by associating them with the point cloud captured by the LiDAR.

Lift: spreading the feature across depth



For each pixel, the network predicts a **visual vector** $f(u)$ and depth weights $p(d_k | u)$, which sum to 1.

$$F(u, d_k) = p(d_k | u) f(u).$$

Each depth receives a copy of the vector, multiplied by its weight. Calibration determines where to place this contribution along the ray.

Scalar example: $f(u) = 2$, with weights of 0.25 at 10 m and 0.75 at 20 m. The contributions are **0.5** and **1.5**.

Phillon and Fidler, ECCV 2020, Figure 3. [Paper and figure source.](#)

Splatting: accumulating into ground-plane cells

The points are expressed in the vehicle frame. For each cell c , we sum **only the contributions that fall into it**:

$$\mathcal{I}_c = \{(u, k) \mid \text{cell}(X_V(u, d_k)) = c\}, \quad F_{\text{BEV}}(c) = \sum_{(u, k) \in \mathcal{I}_c} p(d_k \mid u) f(u).$$

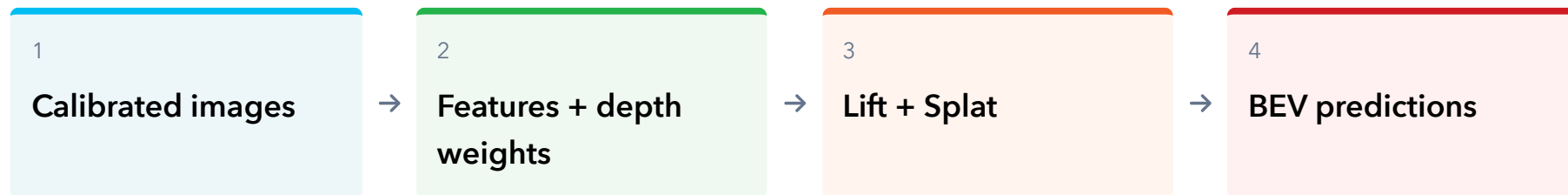
The index (u, k) denotes a pixel and a depth hypothesis. Several pixels or cameras can contribute to the same cell.

A feature grid

The features are not occupancy probabilities. For example, they can be class predictions (pedestrians, cars, etc.).

How to Learn the BEV Representation?

In Lift-Splat-Shoot, a task defined on the ground can supervise the entire chain:



A cell occupied by a vehicle

The annotation says "vehicle". The BEV head assigns it a probability of 0,2: its cross-entropy loss is $-\ln(0,2) \approx 1,61$.

This error backpropagates to the visual features and depth weights that contributed to the grid.

The depth weights can thus be learned **without metric depth annotations**. They are not necessarily calibrated depth measurements.

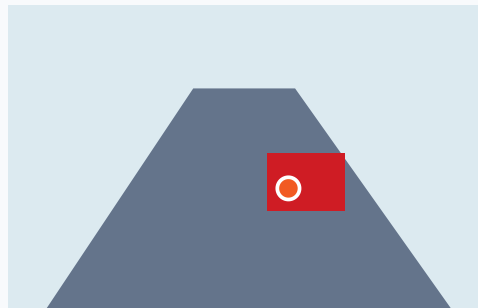
Phillon and Fidler, ECCV 2020. Lift, Splat, Shoot.

Following a Pixel to the BEV Grid

Expérience pédagogique · données synthétiques

Réinitialiser la démonstration

1. Image + caractéristique $f(u)$

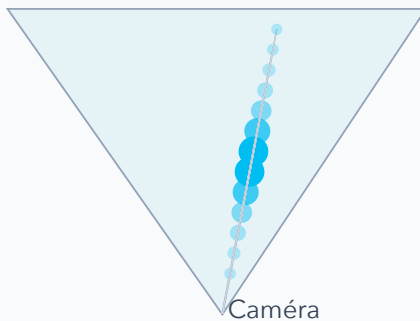


Pixel $u = 190, v = 120$

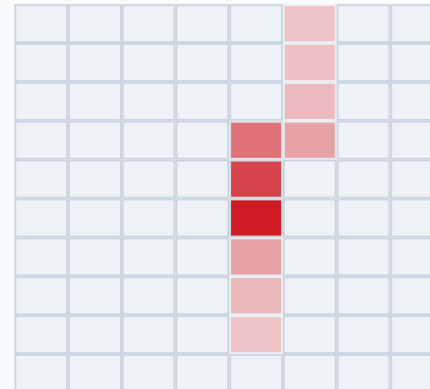
$K : f_x = f_y = 200 \text{ px}$

2. Distribuer sur les profondeurs

Point 3D = $d \cdot K^{-1} \tilde{u}$



3. Accumuler dans la grille BEV



$\Sigma \text{ poids} = 1.000$

Pixel u Profondeur 15 m σ 3 m

Étape

Une caractéristique scalaire $f(u) = 1$ pour lire les poids ; dans le réseau, $f(u)$ est un vecteur.

Change the pixel, the dominant depth and its uncertainty. Follow the displacement and spread in the grid.

What elements should a map represent?

Element	Possible geometry	Useful information
Line / boundary	Polyline	Position and separation
Lane	Center line and boundaries	Width, direction, continuity
Crosswalk	Polygon or boundaries	Crossing zone
Intersection	Connections between lanes	Permitted movements



A curb as a polyline

In the vehicle frame (x forward, y to the left): $[(0, 2), (5, 2), (10, 3)]$ m, class "curb". The points describe its path; the class indicates its meaning for the robot.

Building a local map online

The model estimates the visible elements around the vehicle at each instant. The result can complement an existing map or work without a pre-established local map.

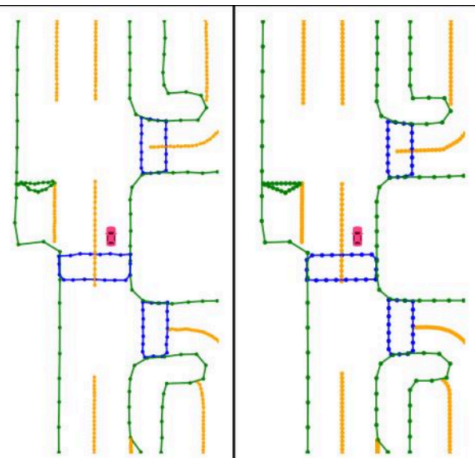
Six camera views of the same scene



■ Pedestrian crossings ■ Lane dividers ■ Road boundaries

Prediction

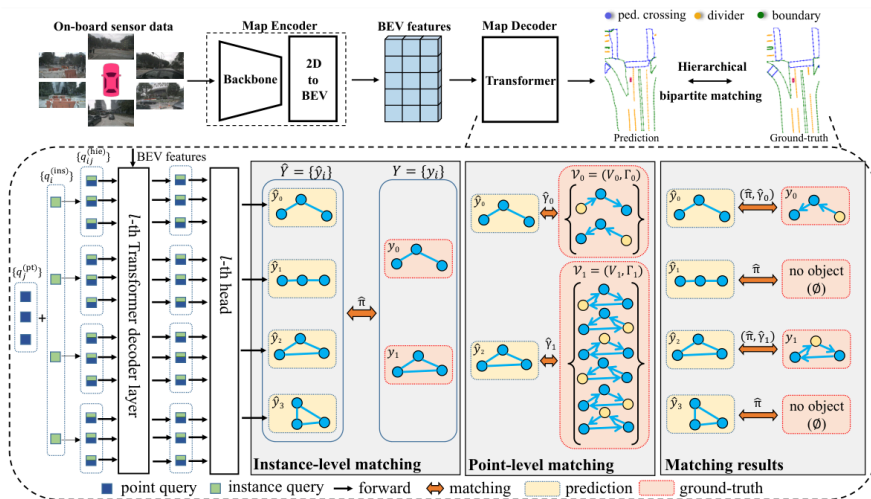
Annotation



For training: the two maps must be compared. The **predicted points and lines** must recover the geometry and class of each annotated element.

Liao et al., MapTR, ICLR 2023, figure 1, first scene cropped. [Paper and figure.](#)

Example: MapTR – predicting vector elements



The model predicts a set of map elements, each described by points.

Training matches predictions to annotations and accounts for valid point permutations. Because the same line read in both directions can represent the same geometry.

Liao et al., MapTR, ICLR 2023, Figure 4. [Paper and figure source.](#)

Defining the loss function for map elements

After matching instances, compare class and geometry. For the **sampled points** of predicted polylines P and annotated Q :

$$d_{\text{Ch}}(P, Q) = \underbrace{\frac{1}{|P|} \sum_{p \in P} \min_{q \in Q} \|p - q\|}_{\text{prediction to annotation}} + \underbrace{\frac{1}{|Q|} \sum_{q \in Q} \min_{p \in P} \|q - p\|}_{\text{annotation to prediction}}.$$

For each point, find its **nearest neighbor**, then average the distances. Redo the computation in the other direction.

A border offset by 20 cm

$P = \{(0, 0), (1, 0)\}$ and $Q = \{(0, 0.2), (1, 0.2)\}$ in meters. The nearest neighbors are all at 0.2 m.

Both averages give $d_{\text{Ch}} = 0.2 + 0.2 = 0.4$ m.

Why compare the polylines in both directions?

A real boundary sampled at $Q = \{0, 1, 2\}$ meters, but predicted only at $P = \{0, 1\}$: part of it is missing.

Direction	Distances to nearest neighbors	Mean
$P \rightarrow Q$	$0 \rightarrow 0: 0; 1 \rightarrow 1: 0$	0
$Q \rightarrow P$	$0 \rightarrow 0: 0; 1 \rightarrow 1: 0; 2 \rightarrow 1: 1$	$1/3 \text{ m}$

The missing part affects the loss function

All predicted points are correct: the first direction penalizes nothing. The second detects that the point at 2 m is not covered. In total, $d_{\text{Ch}} = 1/3 \text{ m}$.

So this favors completeness of the predictions.

Aligning successive observations

An old observation must be expressed in the current frame:

$$X_{V_t} = T_{V_t \leftarrow V_{t-1}} X_{V_{t-1}}.$$



A static point is **10 m ahead of the vehicle**. The vehicle moves forward 2 m: in the new frame, this point is at **8 m**. Without motion estimation, the positions 10 m and 8 m would be wrongly fused. A registration error can thus duplicate a curb.

Building at the scale of a road network



Source	Contribution	Limitation to address
Navigation map / OpenStreetMap	Road structure and approximate connections	Accuracy, completeness, freshness
Aerial imagery	Coverage and global view	Occlusions, resolution, date
Vehicle sensors	Recent road-level detail	Uneven coverage, calibration
Human correction	Arbitration of ambiguous cases	Cost and traceability

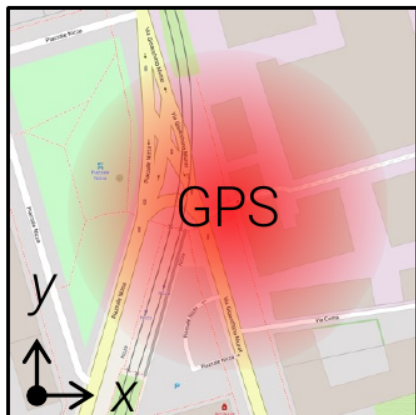
In practice: Mapping a new ramp

The aerial view suggests the layout; several vehicle passes measure the markings and curbs. After alignment, the observations are fused and a human annotator verifies that the map is correct.

Using public maps. E.g.: OpenStreetMap (OSM)

OSM, freely available, is a worldwide road map built from public data and crowdsourcing. It notably describes roads and buildings using **lines, polygons and attributes**, and can therefore serve as a basis for localization or for building an HD map.

OSM map and approximate GPS position



Rasterize
(discretize on
a grid)
by class

Semantic raster



■ Buildings ■ Parks ■ Grass ■ Roads (lines)

It can be useful to use these maps as an (imperfect) basis for long-term planning and to complement them with onboard sensor data for short-term planning. Notably to detect changes, correct the geometry, and detect dynamic objects (pedestrians, cars, etc.).

Sarlin et al., OrienterNet, CVPR 2023, figure 2, two cropped panels. [Paper](#). Data © OpenStreetMap contributors.

Detecting a Persistent Change



When observing the environment, evidence must be accumulated before modifying an HD map.

A missing line may simply be occluded and thus not visible temporarily. A line that appears to have moved may be the effect of a calibration error.

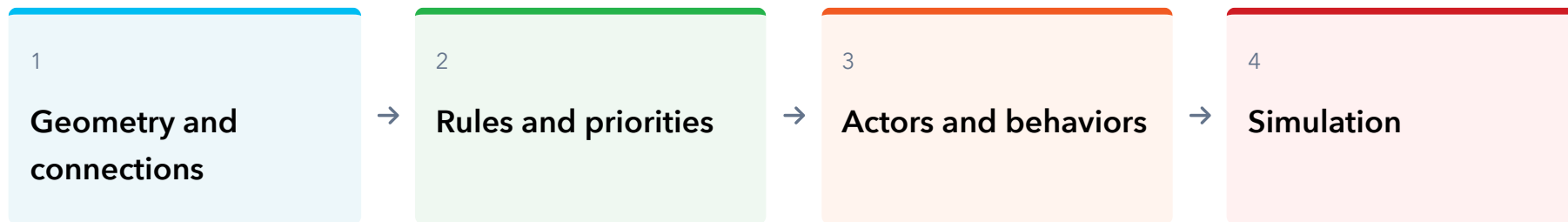
So detecting a persistent (permanent) change in the map requires several consistent pieces of evidence. For example, several vehicle passes noticing the same change.

Traceability

Keep date, provenance, and uncertainty. A validated update must be distinguishable from an isolated observation.

This makes it possible to build highly reliable HD maps that can be shared by a fleet of vehicles. Of course, the process is long and costly, hence the gradual, limited deployments from Waymo, Zoox, and other self-driving car companies.

An abstract map supports multiple scenarios



The maps we build provide a spatial support not only for navigation, but also for simulation.

We can use them to create scenarios by adding vehicles, pedestrians, their states and their behaviors. A map of a single intersection allows simulating a large number of variations.

Abstraction

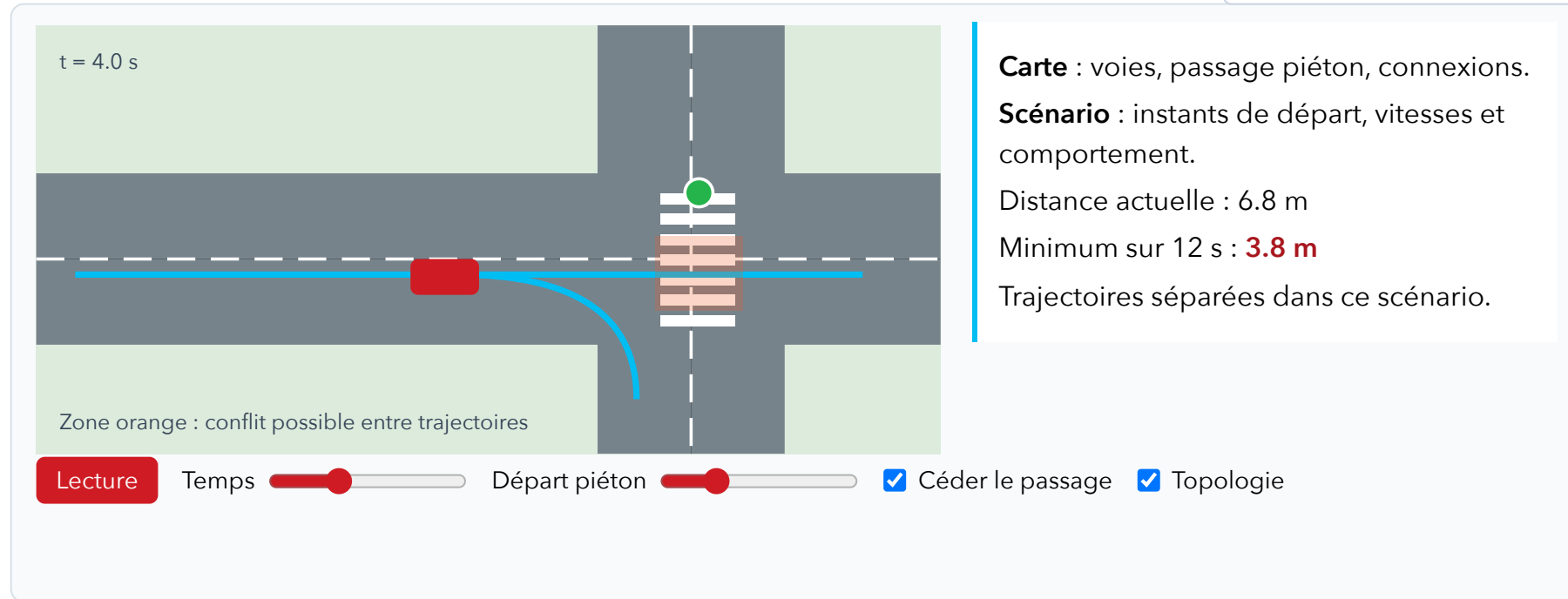
Because the map is just a collection of objects, it is easy to add or remove them to simulate driving scenarios.

Much simpler than generating coherent and realistic camera or LiDAR data for the various scenarios.

Explore an intersection with a pedestrian

Expérience pédagogique · données synthétiques

Réinitialiser la démonstration



Vary the pedestrian's departure time and the yield decision. Compare the trajectories on an identical map.

Multimodal Supervision

Learn with a label source different from the signal used at inference.

Inputs and labels can change modality

Model input	Supervision source	Example task
Image	LiDAR depth	Depth estimation
Image	Pose of a 3D model	Object pose estimation
Image	GNSS positions	Place recognition
LiDAR point cloud	Georeferenced trajectory	3D place recognition

Correspondence between data

The modalities must be synchronized and correctly matched. Their noise carries over to the labels.

Example: Place recognition

We will look at an example: place recognition.

The goal is to replicate a GPS's relocalization capability using only images or LiDAR scans. The learned model can then be used when GNSS (GPS) signals are no longer available.

Recognizing a place seen before

A query image is compared to a database of reference images. Since the images are taken at different times. A challenge is to recognize a place despite changes in lighting, weather conditions, or occlusions.

The result is: a **candidate association** (pair of images), along with a score (confidence/uncertainty).



(a) Mobile phone query

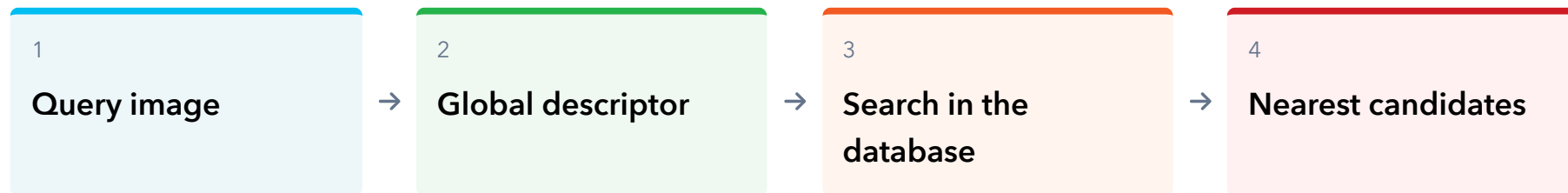


(b) Retrieved image of same place

Arandjelović et al., NetVLAD, CVPR 2016, Figure 1. [Article and figure source.](#)

Place recognition is a search

$$z_q = f_\theta(I_q), \quad z_j = f_\theta(I_j), \quad j^* = \arg \min_j d(z_q, z_j).$$



The global descriptor is a compact vector associated with the entire image. The vectors in the database are computed and indexed ahead of time.

Visual ambiguity

Two distinct places can look alike. Keep several candidates before checking the geometry.

A global descriptor summarizes local characteristics

An encoder produces a local vector $f_i \in \mathbb{R}^D$ at each pixel or patch in the image. Aggregation summarizes these vectors into **one descriptor for the image**.



The descriptor retains the information learned to recognize the place. Its values are not geographic coordinates.

From Retrieval to Loop Closure



A loop closure links two poses of the same graph when a place is revisited. Geometric verification provides a measured relative transformation and its uncertainty.

Measurement scale

A pair of monocular images typically provides a translation at an unknown scale. A complete metric constraint requires additional information.

Example: A robot returns in front of the lab entrance

The robot retrieves an old image of the entrance. Verified RGB-D correspondences make it possible to estimate the relative displacement between the two shots and add the constraint to the graph.

Loop closure residual

$$r_{ij} = \text{Log}(Z_{ij}^{-1}T_i^{-1}T_j) \in \mathbb{R}^6.$$

Quantity	Reading
Z_{ij}	Measured pose from frame j to frame i
$T_i = T_{W \leftarrow i}, T_j = T_{W \leftarrow j}$	Poses of the two cameras in the world
$T_i^{-1}T_j$ and Z_{ij}	Two transformations from frame j to frame i
$Z_{ij}^{-1}T_i^{-1}T_j$	Identity if prediction and measurement coincide
$\text{Log}(\cdot)$	Converts the discrepancy into six local components

Add a loop closure to the graph

Expérience pédagogique · données synthétiques

Réinitialiser la démonstration

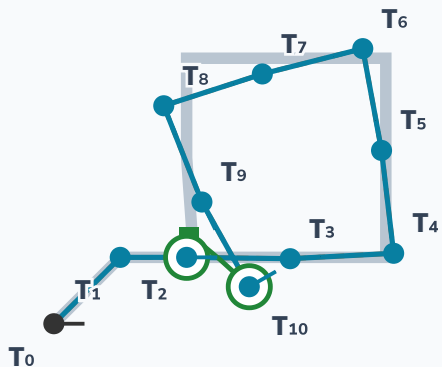
Reconnaissance de lieux

Bonne association · $T_{10} \leftrightarrow T_2$

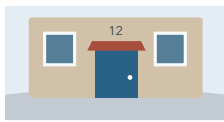


Optimiser

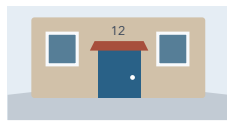
Graphe non optimisé



T₁₀ retrouve T₂ ?



Requête : entrée A



Référence : entrée A

Même lieu, deux passages.

Écart au trajet réel

0,95 → 0,95 m

Coût du graphe

844,2 → 844,2

RMSE des positions · simulation SE(2)

Fermeture fortement pondérée · perte quadratique

T₀ fixé · orientations indiquées par les traits · même échelle dans les deux cas

— Trajet réel — Odométrie initiale — Poses estimées — Fermeture T₂ ↔ T₁₀

Même entrée A : la bonne fermeture est prête. Cliquer sur Optimiser.

Choose a good or a bad association, then click "Optimize" and observe the trajectory.

Using the GNSS position of a retrieved image



A geolocated database provides an approximate position of the recognized place, even if the query has no GNSS.

Approximate anchoring

The query and the reference may have been taken several meters apart. This difference adds to the GNSS noise.

Localizing a robot without GNSS reception

An image from the robot recognizes a facade present in a geolocated database. The position of the reference gives a plausible neighborhood.

Residual of an approximate geographic anchor

With t_i the position of the query camera in a common metric frame:

$$r_i^{\text{geo}} = t_i - p_j^{\text{GNSS}}, \quad E_i^{\text{geo}} = (r_i^{\text{geo}})^\top \Sigma_{\text{geo}}^{-1} r_i^{\text{geo}}.$$

This factor assumes that the positions of the two shots are close enough. Its uncertainty must cover the GNSS error, the shot offset, and association errors.

An anchor in a local frame, in meters

If $t_i = (12, 5, 0)$ and $p_j = (10, 4, 0)$, then $r_i^{\text{geo}} = (2, 1, 0)$ m. With $\Sigma_{\text{geo}} = 4I \text{ m}^2$, the cost is $(2^2 + 1^2)/4 = 1.25$.

How to build a training dataset for place recognition

First, positive and negative pairs can be selected from GNSS positions.

For images geolocated at p_i and p_j :

$$\|p_i - p_j\| < r_+ \Rightarrow \text{possible positive}, \quad \|p_i - p_j\| > r_- \Rightarrow \text{possible negative}, \quad r_- > r_+.$$

An intermediate zone is ignored to reduce ambiguities.
Spatial proximity gives weak supervision: the images may face opposite directions.

Labeling noise

GNSS noise, parallel streets, repetitive facades, and viewpoint changes produce misleading pairs.

Illustrative radii: 10 m and 25 m

At 6 m: possible positive. At 18 m: pair ignored. At 40 m: possible negative. Two images at 6 m can still face opposite facades; proximity alone is not enough.

Easy Negatives, Hard Negatives, and False Negatives

Type	Geographic distance	Descriptor distance	Effect
Easy	Distinct places	Large	Loss is often zero
Hard	Distinct places	Small	Useful learning signal
False negative	Same place or real overlap	Small	Misleading learning signal

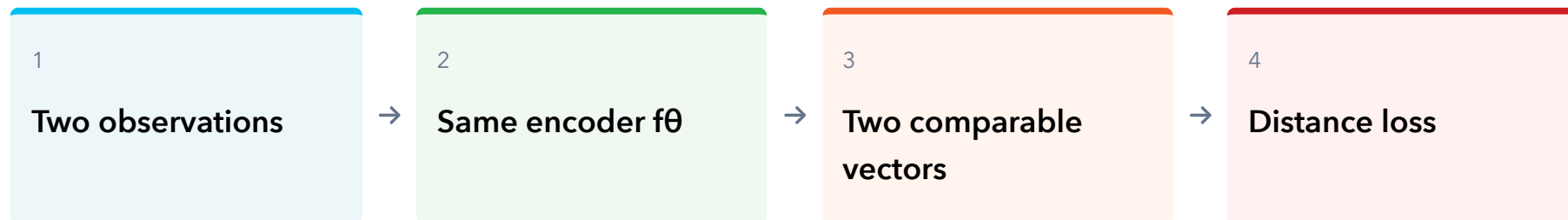


Query: the entrance of a parking lot

Easy negative: a forest. **Hard negative:** the similar entrance of another parking lot. **False negative:** the same entrance, geolocated too far away because of a GNSS error.

How to train: A siamese network

$$z_a = f_{\theta}(I_a), \quad z_b = f_{\theta}(I_b).$$



Both branches use the same parameters. The loss learns a geometry where compatible examples are close.

The same intersection, by day and by night

Each image passes through the same encoder. The positive pair encourages descriptors to stay close despite the lighting; the gradients from both branches update the same weights.

Contrastive loss: learning with pairs

Distance between descriptors: $D = \|z_a - z_b\|_2$. With $y = 1$ for a positive and a margin $m > 0$:

$$\ell_{\text{pair}} = \begin{cases} D^2 & y = 1 \quad (\text{same place}), \\ \max(0, m - D)^2 & y = 0 \quad (\text{different places}). \end{cases}$$

Pair, with $m = 1$	Distance D	Loss	Desired effect
Intersection by day / same intersection at night	0.4	$0.4^2 = 0.16$	Pull descriptors closer
Two different intersections	0.4	$(1 - 0.4)^2 = 0.36$	Push them apart
Two different intersections	1.2	0	Already far enough apart

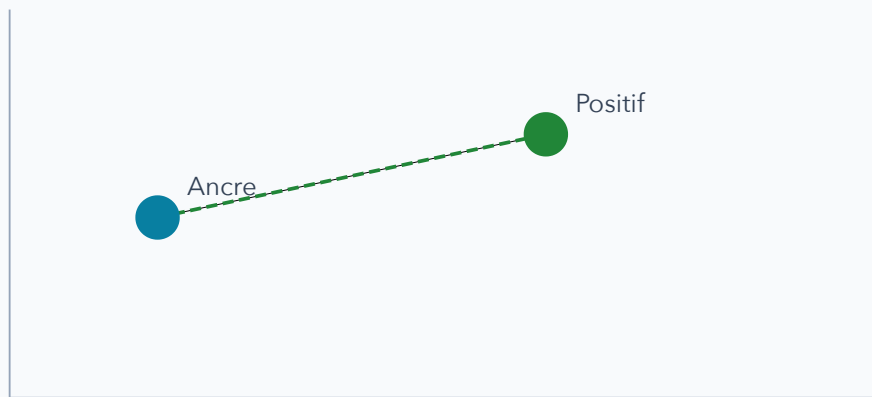
The margin sets a sufficient separation

Negatives beyond m no longer contribute to this loss. D measures a gap in descriptor space, not a distance in meters between places.

Bring positives closer, push negatives apart in descriptor space

Expérience pédagogique · données synthétiques

Réinitialiser la démonstration



Paire positive

$d = 2.184$

$\ell = 4.770$

Rapprocher les deux représentations.

Déplacer les points, puis appliquer un pas de gradient.

Pas de gradient

Marge



1.0



Paire positive

0 mise(s) à jour

Change the pair type and the margin, then apply gradient steps.

Triplet loss: enforcing a relative margin

Anchor a : intersection at night; positive p : same intersection during the day; negative n : a different intersection.

$$\ell_{\text{tri}} = \max \left(0, \underbrace{\|z_a - z_p\|_2}_{d_{ap}} - \underbrace{\|z_a - z_n\|_2}_{d_{an}} + m \right).$$

We want $d_{an} \geq d_{ap} + m$: the negative must be farther than the positive **by at least the margin**.

$d_{ap} = 0.4, m = 0.2$	Negative distance d_{an}	Loss
Negative farther, but margin insufficient	0.5	$0.4 - 0.5 + 0.2 = 0.1$
Margin satisfied	0.7	$\max(0, -0.1) = 0$

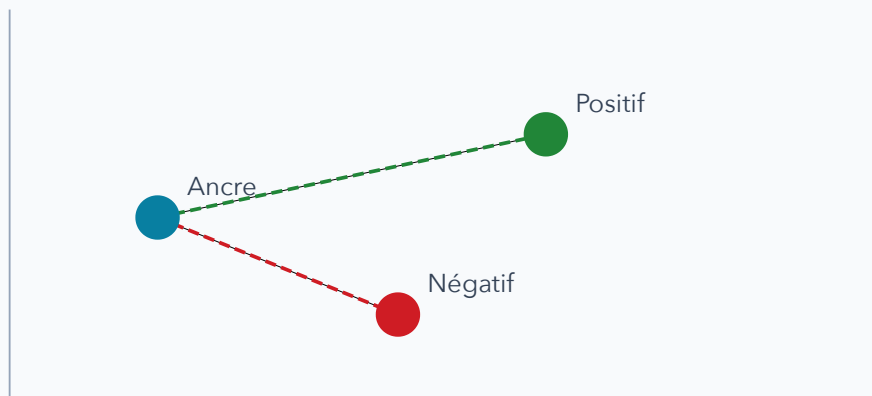
Advantage

The triplet loss function is more stable. We avoid pulling descriptors closer and then pushing them apart successively. We do both at the same time.

See the Effect of Triplet Loss

Expérience pédagogique · données synthétiques

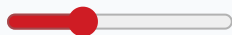
Réinitialiser la démonstration



Déplacer les points, puis appliquer un pas de gradient.

Pas de gradient

Marge



1.0 0 mise(s) à jour

$$d(a,p) = 2.184$$

$$d(a,n) = 1.476$$

$$\ell = \max(0, d(a,p) - d(a,n) + m)$$

$$\ell = 1.708$$

La contrainte de marge est violée.

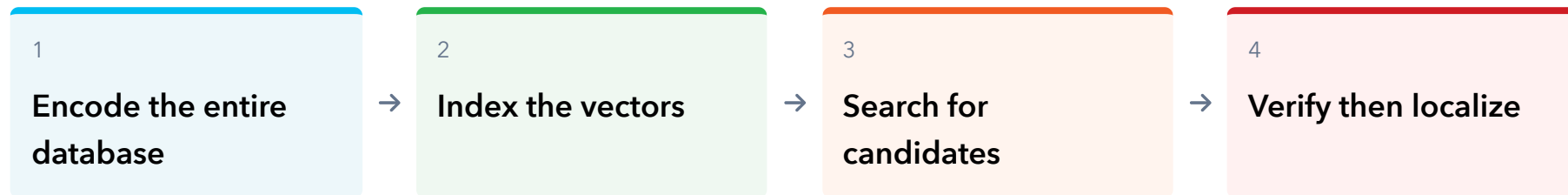
Move the anchor, positive, and negative. Adjust the margin, then follow the updates of the three points.

The training and retrieval pipeline

Pipeline: training:



Pipeline: retrieval:



Visual conditions can vary widely



Day/night, weather, seasons, structure, and viewpoint change appearance.

A good evaluation should separate these factors and include examples that are genuinely different from the training data.

Warburg et al., Mapillary SLS, CVPR 2020, Figure 2. [Paper and figure source.](#)

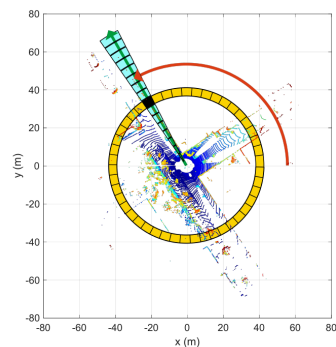
Learning targeted robustness

Variation	Concrete approaches	Evaluation
Seasons	Multi-temporal positives; stable local features	Independent summer/winter revisits
Weather	Diversified collections; photometric augmentations and plausible degradations	Real rain/fog sequences
Time of day	Day/night pairs; training with strong lighting variations	Held-out nighttime queries
Viewpoint	Multi-view positives, EigenPlaces; local geometric verification	Lateral displacements and opposing views

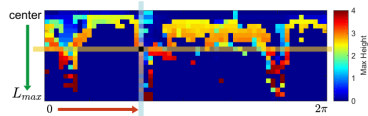
Limitation

A synthetic augmentation helps cover a variation; it does not replace evaluation on real observations.

Scan Context: LiDAR Place Recognition



(a) Bin division along azimuthal and radial directions



(b) Scan context

The horizontal plane is divided into rings and sectors.

Each cell keeps a maximum height, producing a matrix that describes the structure around the sensor.

The original descriptor is built explicitly, without supervised training.

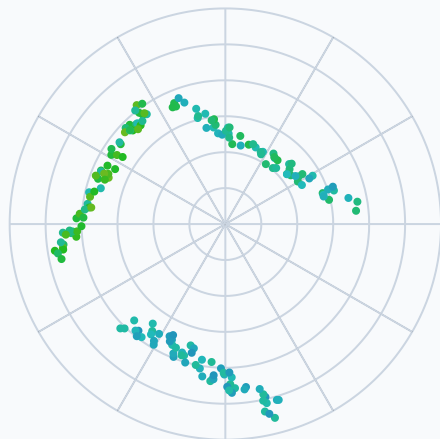
Kim and Kim, Scan Context, IROS 2018, Figure 1. [Paper](#) and [figure source](#).

Building and aligning a polar descriptor

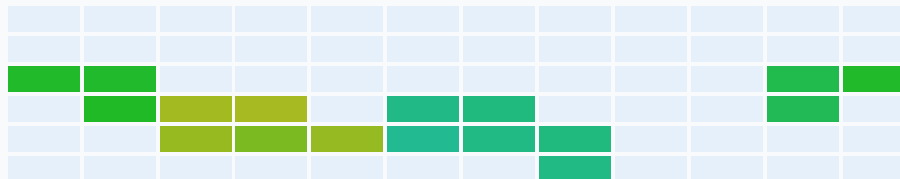
Expérience pédagogique · données synthétiques

Réinitialiser la démonstration

Nuage et secteurs polaires

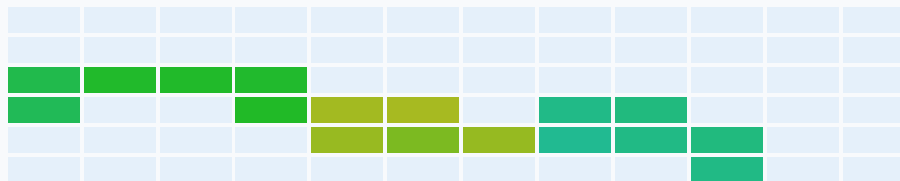


Référence : hauteur maximale / cellule



Colonnes : secteurs · lignes : anneaux

Requête après décalage circulaire



Rotation LiDAR



60°

Décalage



0

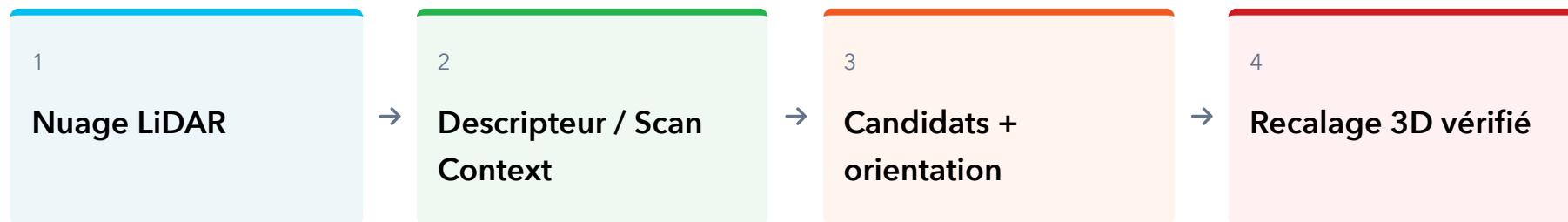
Aligner

Distance = 0.395

Une rotation devient un décalage de colonnes. La translation et les occultations modifient aussi le contenu des cellules.

Rotate the point cloud, then search for the column shift that minimizes the distance.

LiDAR Loop Closure: From Candidate to Constraint



LiDAR provides geometry that is less dependent on lighting. Structural changes, occlusions, and lateral shifts remain difficult.

Synthesis

Connect supervision, representation, geometry, and robotic decision-making.

Designing a Supervised System

Question	Examples from the course
What target is actually available?	Class, box, pose, polyline, GNSS position
What representation serves the task?	Probabilities, object volume, map, descriptor
What loss expresses the relevant error?	Cross-entropy, regression, geometry, margin
What check protects robotic use?	Independent validation, visibility, geometry, uncertainty

Garbage In Garbage Out

In supervised learning, the crux of the matter is the quality of the training data. Without good data, you get bad results.